CCPROG3 MCO Project Specifications
One Piece Universe Management System
John Byron Tuazon_S22A_S22B

This document outlines the requirements and design for the **One Piece Universe Management System (OPUMS)**, a Java application modeling the factions, supernatural elements, and socio-economic dynamics of the *One Piece* world.

To successfully model the One Piece Universe Management System (OPUMS), your design must leverage core Object-Oriented Programming principles to handle the following domain requirements:

- Shared and Unique Traits: The system comprises various individuals, specifically Pirates, Marines, Pirate Hunters, and Civilians. All of these individuals share common baseline data and logic, such as checking if they have enough money in their wallet. However, your design must reflect that each specific type of individual also maintains its own unique data and operations.
- Dynamic Profile Displays: The system must be able to trigger a profile display for any individual. The output must dynamically adapt to the individual's specific categorization. For example, requesting a Pirate's profile should reveal their bounty and crew, while requesting a Marine's profile should reveal their rank and corps.
- Role-Specific Duties: Every individual must be able to perform the duties relevant to their role. The execution of this action varies depending on the entity. For instance, a Pirate acting as a "Cook" might output "Cooking for the crew...", whereas a Marine might output "Patrolling the seas...".
- Data Integrity and Validation: The internal state of all entities must be strictly protected from direct external modification. You must implement controlled access mechanisms for data retrieval and modification, ensuring that any changes are structurally validated before being applied (for example, the system must definitely prevent a bounty from being set to a negative value).
- Entity Specificity: In this universe, an individual cannot exist merely as a generic, undefined entity. Every individual introduced into the system must be strictly categorized and created as a specific entity, such as a Pirate or a Civilian.

I

Module 1: Character Database

This module serves as a centralized character database, dynamically categorizing characters into Pirates, Marines, Pirate Hunters, and Civilians. Utilizing object-oriented inheritance, these specific sub-factions inherit core attributes and behavioral methods from a unified parent class. Every character's profile can be mapped to their factual attributes in the source material (anime or manga).

A key constraint dictates that any character specifically categorized as a pirate must be assigned an individual bounty. Each character must only belong to one pirate crew (if the character is a pirate) or one Marine Corps (if the character is a Marine).

Module 1 Functionalities:

1. Add a new character
2. View a character
3. Modify a character
4. Delete a character

Universal Character Details

All characters in the database, regardless of their faction, must maintain the following baseline data and capabilities:

Attributes (baseline but not limited to):

- **Character ID:** Unique ID of all characters in the database. This must be auto-generated.
- **Name:** Name of the character ("Luffy", "Zoro", "Sanji", etc.)
- **Alias:** Alias of the character ("Straw Hat", "Pirate Hunter", "Black Leg", etc.)
- **Origin:** Home origin of the character ("East Blue", "West Blue", "Alabasta", etc.)
- **Status:** Current state of the character ("Free", "Captured", "Dead")
- **Devil Fruit Power:** Indicates the character's Devil Fruit (leave blank by default)
- **Wallet:** Current money in "Berries" of the character

Methods (baseline but not limited to):

- **Display Profile:** Displays the character's comprehensive profile.

Pirate Details

Attributes (baseline but not limited to):

- **Bounty:** Current bounty of the pirate character in Berries
- **Pirate Role:** Current role of the pirate in the crew ("Captain", "Navigator", "Cook", etc.)
- **Is Captain:** Boolean flag to indicate if the member is the captain or not
- **Pirate Crew:** The current crew that the pirate belongs with

Methods (baseline but not limited to):

- **Assign/Modify Bounty:** Allows for the assignment or modification of their bounty
- **Assign to Pirate Crew:** Assign or change the crew alliance of the pirate

Marine Details

Attributes (baseline but not limited to):

- **Rank:** The current rank of the marine ("Fleet Admiral", "Admiral", "Vice Admiral", etc.)
- **Marine Corps:** Current Marine unit assigned to

Methods (baseline but not limited to):

- **Promote Rank:** Changes the rank of the marine
- **Assign to the Marine Corps:** Assign a Marine to a corps

Pirate Hunter Details

Attributes (baseline but not limited to):

- **Combat Style:** Primary combat style of the pirate hunter ("Three-Sword Style", "Sniper", "Brawler", etc.)
- **Confirmed Captures:** Total number of pirates turned over to the world government

Civilian Details

Attributes (baseline but not limited to):

- **Profession:** ("Shipwright", "Bartender", "Scholar", etc.)
- **Residence:** ("Water 7", "Whisky Peak", "Syrup Village", etc.)

Module 2: Affiliation Database

This module governs the collective organization of characters into organized factions, allowing them to form either a Pirate Crew or a Marine Corps unit. To maintain system integrity, strict structural validation ensures that each distinct character can be assigned to only one group at any given time. For Pirate Crews, the system dynamically manages a cumulative 'Total Crew Bounty' attribute. The module monitors the life status of all constituent pirates; if a pirate is captured or confirmed deceased, they cease to contribute to the crew's total active bounty valuation.

Module 2 Functionalities:

1. Create Group (Pirate Crew or Marine Corps)
2. View Groups (Once listed, there should be a way to view the members and their individual profiles)
3. Edit group attributes
4. Add members
5. Remove members

Pirate Crew Details

When establishing and managing a pirate crew, the system must track and manipulate the following domain requirements:

Attributes: (baseline but not limited to):

- **Crew ID:** Unique identifier for the group
- **Crew Name:** The pirate crew's popular name ("Straw Hats", "Foxy Pirates", etc.)
- **Ship's Name:** The name of the crew's flagship ("Going Merry", "Thousand Sunny", etc.)
- **Captain's Name:** The designated leader of the crew
- **Crew Members:** The collection of individual characters belonging to this crew

- **Total Crew Bounty:** The calculated sum of bounties of all active crew members in Berries. Crew members who are already captured or dead must be excluded from this calculation.

Methods: (baseline but not limited to):

1. **Group Operations:** The system must allow the crew to recruit new pirate members and discharge or remove existing ones.

Marine Corps Details

When establishing and managing a Marine unit, the system must track and manipulate the following domain requirements:

Attributes: (baseline but not limited to):

- **Corps ID:** Unique identifier for the Marine unit
- **Base Location:** The designated Marine Corps Headquarters ("Marine Ford", "Enies Lobby", etc.)
- **Corps Commander:** The designated leader of the unit
- **Operational Funds:** Financial resources available to the corps

Methods: (baseline but not limited to):

- **Group Operations:** The system must allow the unit to recruit new Marine members and discharge or remove existing ones.

Module 3: Devil Fruit Database Module

This module serves as an inventory-tracking registry for all mythical Devil Fruits, recording critical data points, such as the fruit's class and category, along with its current and historical ownership lineage. The system maintains absolute exclusivity: a single Devil Fruit can be assigned to only one character at any given timestamp. All modifications, transformations, and transfers of fruit ownership must be explicitly executed within this dedicated registry module. In the event of a character's death, an automated system trigger immediately marks the Devil Fruit as available for world redistribution.

Module 3 Functionalities:

1. Create devil fruit
2. View devil fruit (including its historical users)

3. Assign to a new user

Devil Fruit Details

When registering and tracking a Devil Fruit, the system must maintain the following domain data:

Attributes: (baseline but not limited to):

- **Fruit ID:** Unique identifier for the fruit.
- **Fruit Name:** The specific name of the fruit (e.g., "Gomu Gomu No Mi", etc.).
- **Category:** The classification type ("Paramecia", "Zoan", "Logia", etc.).
- **Primary Ability:** A description of the traits and powers granted to the user.
- **Current Owner:** The individual currently in possession of the fruit's power.
- **Historical Owners:** A sequential record of all past individuals who previously held this fruit's power.

Methods: (baseline but not limited to):

- **Assign to a New User:** The process of transferring the fruit's power to a living character, provided there is no current owner.
- **Trigger Reincarnation:** The automated process that occurs upon the current owner's death, which releases the fruit, adds the deceased to the historical owners list, and marks it as available for world redistribution.

Everything you need to know about Devil Fruits can be found here:
https://onepiece.fandom.com/wiki/Devil_Fruit

Module 4: Bounty System

This module provides processing infrastructure for the capture and surrender of pirates. It permits eligible factions, specifically Pirate Hunters, Marines, and Civilians, to turn over wanted pirates to the authorities. The operational pipeline provides a binary operational flag indicating whether the target pirate is 'Dead' or 'Alive'. Upon a successful turn-in transaction, the system dynamically increments the captor's financial asset reserve with the target's reward money, adjusts the captured target's state, and applies the corresponding financial deductions to the victimized pirate crew's combined total bounty valuation. If marked as “Alive”, the pirate will still be part of the crew but will have a “Captured” status.

Module 4 Functionalities:

1. Register a capture (recording both the captured pirate and the captor)
2. View historical captures (a historical data log of all successful capture events)

Capture Record Details

When a capture is officially registered, the system must generate and maintain a record containing:

Attributes: (baseline but not limited to):

- **Capture ID:** A unique identifier for the transaction.
- **Captured Pirate:** The individual who was apprehended.
- **Captor:** The individual who executed the capture.

Methods: (baseline but not limited to):

- **Validate Captor:** Check the acting captor's affiliation to prevent illegal bounty claims.
- **Process Target Status:** Execute the pipeline that updates the victim's life status, modifies their crew's total bounty, and releases their Devil Fruit if applicable.
- **Route Financial Rewards:** Distribute the bounty funds to the appropriate destination based on the captor's faction.
- **Log Transaction:** Finalize and record the capture event in the system's history.

Other Restrictions and Rules

To ensure system integrity, your database controllers must enforce these strict operational constraints:

Affiliation & Character Restrictions

1. **Exclusivity of Factions:** A character cannot be a Pirate and a Marine simultaneously.
2. **Single Group Allegiance:** A Pirate can only belong to one Pirate Crew at a time; a Marine can only belong to one Marine Corps at a time.
3. **Command Validation:** A Pirate Crew must have exactly one Captain. If a new Captain is assigned, the previous Captain's pirate Role must automatically revert to a standard role (e.g., "Crew Member").

Devil Fruit Exclusivity Rules

1. **The "One Fruit, One Soul" Rule:** A Devil Fruit can have only one current owner at a time. The fruit can only be assigned if there is no current owner. The current owner can only relinquish the devil fruit if he/she is marked as “dead”.
2. **The Reincarnation Trigger:** When a character's status is modified to "Dead", an observer pattern or system trigger must immediately execute:
    - Append the deceased character to the fruit's historical owners list.
    - Set Current Owner to null.
    - Clear the Devil Fruit Power field on the deceased character's profile.

Bounty & Financial Transaction Safeguards (Module 4)

1. **The Dead or Alive Pipeline:** When registering a capture:
    - If marked **"Alive"**: The target's status updates to "Captured". They remain in the crew, but their bounty is deducted from the Total Crew Bounty.
    - If marked **"Dead"**: The target's status updates to "Dead". They are automatically removed from their PirateCrew entirely, and their Devil Fruit is reincarnated.
2. **Financial Reward Routing:** The bounty for the captured pirate must be automatically added to the captor's wallet.
3. **Illegal Captures:** A Pirate cannot capture another pirate and claim a bounty through official channels (only Pirate Hunter, Marine, and Civilian are valid captor types). If the captor is a Marine, the money should go to the Marine Corps operational fund rather than their personal wallet.

Milestones

1. MCO1 - due on Jul 11, 2026 (21:00)
    a. Draw the UML class diagram to represent the system described.
    b. Implement all classes and methods from Modules 1 - 3.
    c. Database/File processing is not yet required.
2. MCO2 - due on Aug 3, 2026 (07:30)
    a. You must follow the MVC architecture.
    b. Your final system must be a GUI-based program. All interactions will be on the GUI components. Console outputs will not be checked.
    c. Complete the UML class diagram.
    d. Phase 2 implementation requires a GUI.
    e. Additional features may be implemented by the programmer. However, these should not contradict the requirement.
    f. Implemented additional features should be reflected in the UML class diagram as well.

Deliverables

1. The design and implementation of the solution should:
    a. Conforms to the specifications described above
    b. Exhibit proper object-oriented and object-based concepts, such as encapsulation and information hiding.
    c. NOT be derived or influenced from any use of Generative AI tools or applications
2. To make it easier to validate the program, the use of libraries outside the Java 21 API is not allowed.
3. Signed declaration of original work (declaration of sources and citations may also be placed here)
    a. See Appendix A for an example
4. Softcopy of the class diagram following UML notations (in PDF or PNG)
    a. Ensure that the diagram is easy to read and well-structured
5. Javadoc-generated documentation for proponent-defined classes with pertinent information
6. Zip file containing the source code with proper internal documentation
    a. The program must be written in Java
7. Test script following the format indicated in Appendix B

    a. In general, there should be at least 3 test case categories (as indicated in the description) per method (except for setters and getters).
    b. There is no need to test user-defined methods that are ONLY for screen design (i.e., no computations/processing; just print/println).
8. For MCO1 only: A video demonstration of your program (max 10 minutes)
    a. While groups are free to conduct their demonstration, a demo script will be provided closer to the due date to help demonstrate the expected functionalities.
    b. The demonstration should also quickly explain key aspects of the program's design found in the group's class diagram
    c. Please keep the demo as concise as possible and refrain from adding unnecessary information
    d. Video demonstration should include a voice-over explanation (not music background nor pure silence)
9. The student (s) should make pertinent backups of their own project. A softcopy of the final, unmodified files (for each phase) should be sent to the student's own email address (es), in addition to regular progress submissions in AnimoSpace and/or Git.

Submission

All deliverables for the MCO are to be submitted via AnimoSpace. Submissions made in other venues will not be accepted. Please also make sure to take note of the deadlines specified on AnimoSpace. No late submissions will be accepted.

Grading

For grading of the MCO, please refer to the MCO rubrics indicated in the syllabus.

Collaboration and Academic Honesty

This project is meant to be worked on in pairs (i.e., a max of two members per group). In exceptional cases, a student may be allowed by their instructor to work on the project alone; however, permission should be sought as collaboration is a key component of the learning experience. Under no circumstances will a group be allowed to work on the MCO with more than two members.

A student cannot discuss or ask about design or implementation with anyone other than the teacher and their groupmate. [Questions about the MP specs should be raised in the Discussion page in AnimoSpace. Copying other people's work and/or working in collaboration with other

teams are not allowed and are punishable by a grade of 0.0 for the entire CCPROG3 course and a case may be filed with the Discipline Office. In short, do not risk it; the consequences are not worth the reward. Comply with the policies on collaboration and AI usage as discussed in the course syllabus.

Documentation and Coding Standards

Do not forget to include internal documentation (comments) in your code. At the very least, there should be an introductory comment and a comment before every class, every method, and every attribute. This will be used later to generate the required External Documentation for your Machine Project via Javadoc. You may use an IDE or the appropriate command-based instructions to create the documentation, but it must be PROPERLY constructed.

Please note that we're not expecting you to add comments for each and every line of code. A well-documented program also implies that coding standards are adhered to in a way that supports code documentation. Comments should be considered for more complex logic.

Bonus Points

No bonus points will be awarded for MCO1. Bonus points will only be awarded for MCO2. The above description of the program is the basic requirement. Any additional features will be left to the student's creativity. Bonus points would be awarded based on the additional features implemented. However, make sure that all the minimum requirements are fully met and correctly implemented first; if not, no additional points will be credited, despite the additional features. To encourage the use of version control, please note that a small portion of the MCO2 bonus points will be awarded for its use. Consider using version control as early as MCO1 to facilitate collaboration within the group.

Resources and Citations

All sources should have proper citations. Citations should be written in APA style. Examples of APA-formatted citations can be seen in the References section of the syllabus. You're encouraged to use the declaration of original work document as the document to place the citations.

Further, this is to emphasize that you DO NOT need to create any original assets, such as pictures and audio, for the application. You can use what's available on the Internet in your project, just make sure to cite your sources.

Demo

Demo for MCO1 is via a video submission. All members are expected to attend the video demonstration and to have roughly equal speaking time during the discussion. Any student who is not present during the demo will receive a zero for the phase.

In MCO2, the demo is live and will include an individual demo problem. The demo will generally be during class time, but the faculty may open other times if there is not enough time to accommodate everyone. The sequence (of who goes first in the class to do the demo) is determined by the faculty. Thus, do not be absent or late during announced demo days. A student or a group that is not present during the demo or who cannot answer questions regarding the design and implementation of the submitted project convincingly will incur a grade of 0 for that project phase.

During the MP demo, it is expected that the program can be compiled successfully in the command prompt and will run. If the program does not run, the grade for that phase is 0. However, a running program with complete features may not necessarily get full credit, as design and implementation (i.e., code) will still be checked.

Other Notes

You are also required to create and use methods and classes whenever possible. Make sure to use Object-Based (for MCO1) and Object-Oriented (for MCO2) Programming concepts properly. No brute force solution.

Statements and methods not taught in class can be used in the implementation. However, these are left for the student to learn on their own.

Appendix A. Template for Declaration of Original Work

Declaration of Original Work

We/I, [Your Name(s)] of section [section], declare that the code, resources, and documents that we submitted for the [1st/2nd] phase of the major course output (MCO) for CCPROG3 are our own work and effort. We take full responsibility for the submission and understand the repercussions of committing academic dishonesty, as stated in the DLSU Student Handbook. We affirm that we have not used any unauthorized assistance or unfair means in completing this project.

[*In case your project uses resources, like images, that were not created by your group.*] We acknowledge the following external sources or references used in the development of this project:

1. Author. Year. Title. Publisher. Link.
2. Author. Year. Title. Publisher. Link.
3. Author. Year. Title. Publisher. Link.

By signing this declaration, we affirm the authenticity and originality of our work.

Signature and date	*Signature and date*
Student 1 Name	Student 2 Name
ID number	ID number

[*Note to students: Do not submit documents where your signatures are easily accessible. Ideally, submit a flattened PDF to add a layer of security for your digital signatures*]

Appendix B. Example of Test Script Format

Class: MyClass						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/ F**
isPositive	1	Determines that a positive whole number is positive	74	true	true	P
	2	Determines that a positive floating point number is positive	6.112	true	true	P
	3	Determines that a negative whole number is not positive	-871	false	false	P
	4	Determines that a negative floating point number is not positive	-0.0067	false	false	P
	5	Determines that 0 is not positive	0	false	false	P

Source of information:

https://onepiece.fandom.com/wiki/One_Piece_Wiki